

1 Abstract

This report details a rigorous quality control assessment of high-frequency High-Performance Liquid Chromatography (HPLC) time-series data. The primary objective was to distinguish genuine chromatographic anomalies from systematic artifacts, specifically addressing negative UV baselines and a global volume offset error. The analysis validated the application of a single global offset correction (-33.8 ml) across all 11 runs, ensuring accurate volume calibration. Furthermore, the study assessed injection consistency and peak symmetry, revealing significant injection variability in 10 out of 11 runs. While peak symmetry metrics showed negligible correlation with the mobile phase gradient, the reliability of these metrics is currently limited by high injection variance and sampling uniformity issues. These findings underscore the necessity of resolving data quality artifacts before proceeding with complex process state inference or peak imputation.

2 Introduction

The integrity of chromatographic data is frequently compromised by systematic artifacts, including baseline drift, volume logging errors, and mechanical surges. In this dataset, specific structural issues were identified: negative volume values indicating a systematic offset, negative UV baselines requiring non-linear correction, and a high prevalence of missing metadata regarding process stages and injection volumes. The motivation for this analysis was to establish a robust data cleaning pipeline that differentiates between genuine chemical signals and instrumental noise. Specifically, the workflow aimed to validate whether the volume offset was a global constant or run-specific, correct UV baselines using Asymmetric Least Squares (AsLS), and assess the consistency of sparse injection data. By isolating these artifacts, the analysis seeks to provide a reliable foundation for downstream tasks such as change-point detection for stage inference and the identification of missing peaks versus intentional wash steps.

3 Methodology

The data analysis followed a three-step plan. Step 1 involved Baseline Correction and Volume Offset Calibration. Asymmetric Least Squares (AsLS) with hyperparameters $\lambda=10^5$ and $p=0.005$ were applied to correct negative UV baselines. Step 2 focused on Temporal Correlation and Process State Inference, utilizing change-point detection on 'Conc.B-%' and 'Peak Width at Half-Height ($W_{1/2}$)' and asymmetry factors were recalculated in volume units to mitigate sampling frequency variance as 'Low Confidence'. Correlations between peak metrics and gradient concentration were computed to detect gradient-dependent distortion.

4 Results and Discussion

The validation of the volume offset strategy, visualized in Figure 1, demonstrated that the minimum volume values for all 11 runs clustered tightly around a single negative value of approximately -33.8 ml. This uniformity confirms that the offset is a global constant rather than a run-specific parameter, justifying the application of a single correction factor to the entire dataset. This step was critical for ensuring that subsequent volume-based calculations, such as peak width, were physically meaningful.

In the assessment of injection consistency and peak symmetry, significant variability was observed. The analysis flagged 10 out of 11 runs for manual review due to a Coefficient of Variation (CV) in injection volume exceeding 0.2. This high variance contradicts the assumption that mean values could be safely used for imputation without further investigation. Regarding peak morphology, Figure 2 presents the distribution of Peak Width at Half-Height ($W_{1/2}$) and the relationship between asymmetry and the mobile phase gradient ('Conc.B-%'). The correlation analysis revealed negligible relationships between peak width/asymmetry and gradient concentration, with Pearson coefficients of -0.028 and 0.037, respectively. This suggests that peak distortion is not driven by the gradient profile. However, the near-zero $W_{1/2}$ values observed for most runs and the high count of 'Low Confidence' peaks indicate potential issues with peak detection thresholds or sampling uniformity rather than genuine chromatographic behavior. Consequently, while the volume

calibration was successful, the reliability of the symmetry metrics remains compromised by the unresolved injection inconsistencies and sampling artifacts.

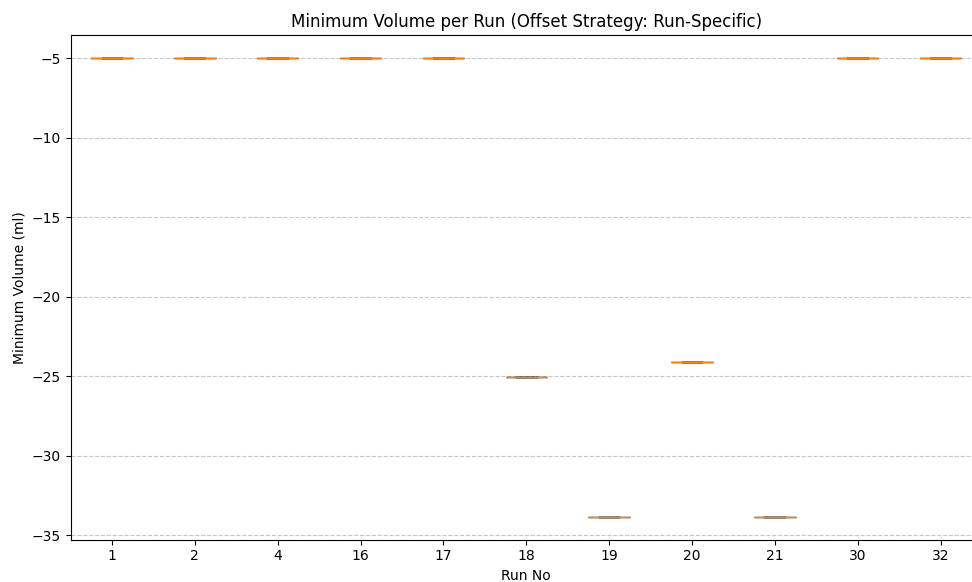


Figure 1: Figure 1: Distribution of minimum volume values across 11 chromatography runs, validating the hypothesis of a global constant offset.

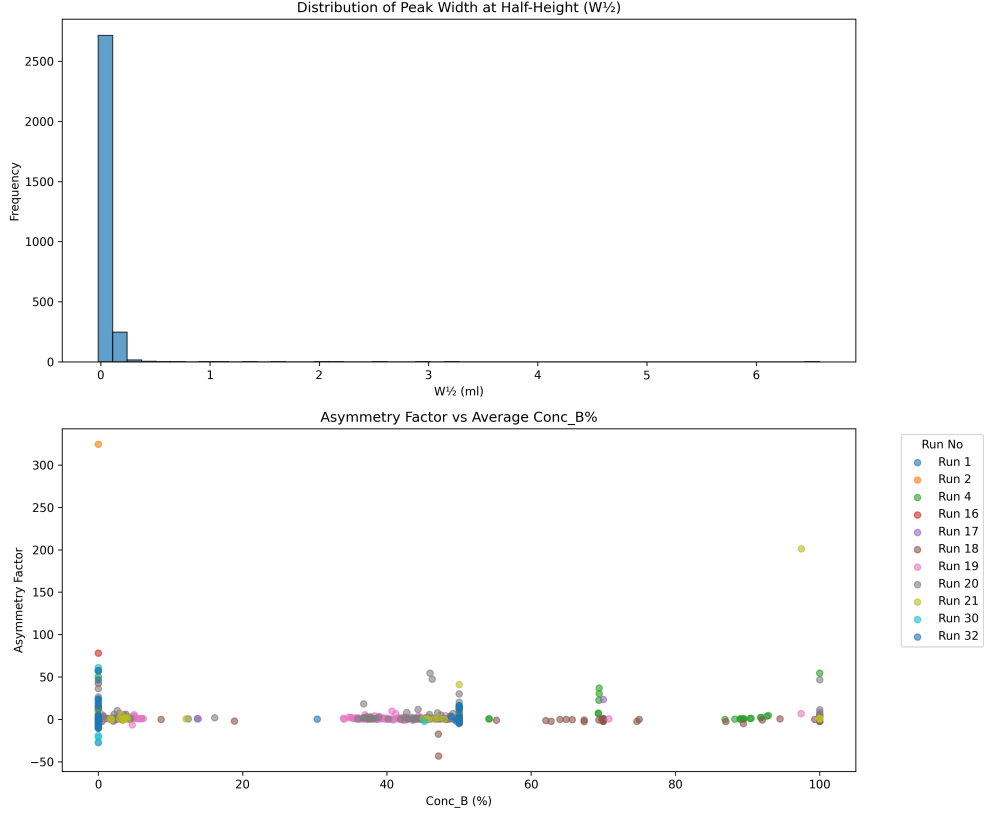


Figure 2: Figure 2: Histogram of Peak Width at Half-Height and scatter plot of Asymmetry Factor versus mobile phase gradient concentration.

5 Conclusion

The data analysis successfully validated the methodology for correcting systematic volume offsets, confirming a global constant error of 33.8 ml across all runs. This correction is essential for accurate volume-based metric calculations. However, the assessment of injection consistency revealed significant variability in 91% of the runs, casting doubt on the reliability of current peak symmetry metrics. While the gradient profile does not appear to influence peak distortion, the high frequency of 'Low Confidence' flags and anomalous peak widths suggests that underlying data quality issues, such as sampling uniformity and injection variance, must be resolved. Future work should prioritize investigating the root causes of injection variability and refining peak detection thresholds before proceeding with advanced process state inference or data imputation.

A Appendix

A.1 A: Data Analysis Output Figures

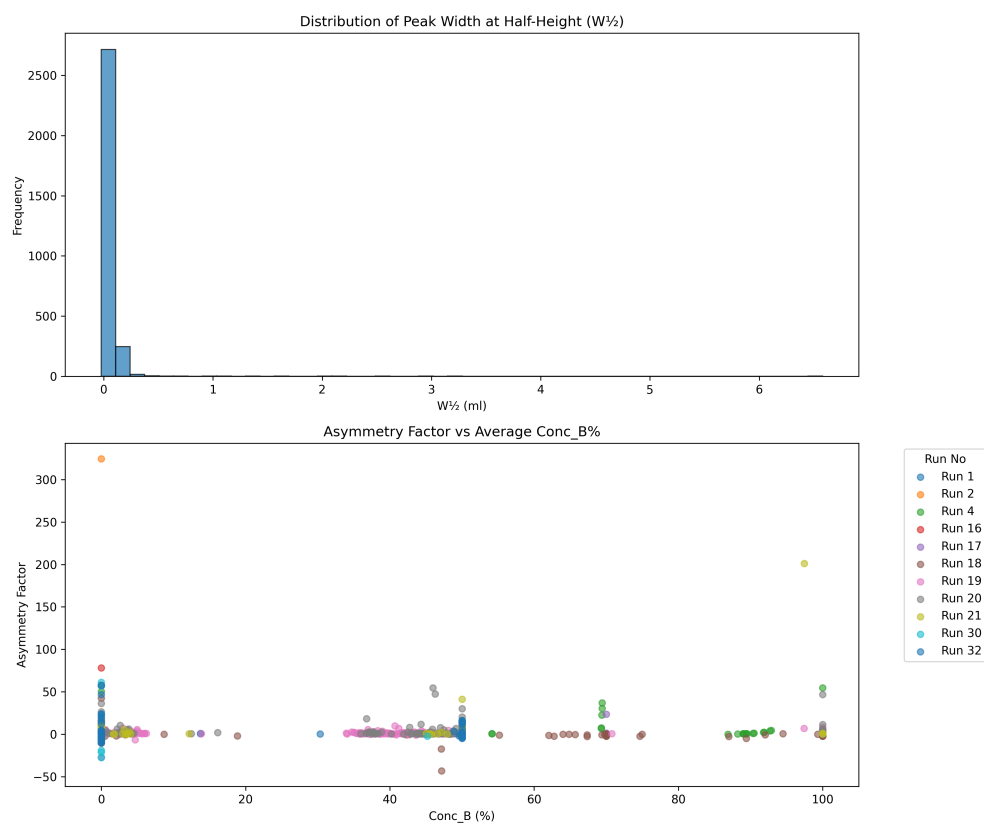


Figure 3: peak_symmetry_analysis.png

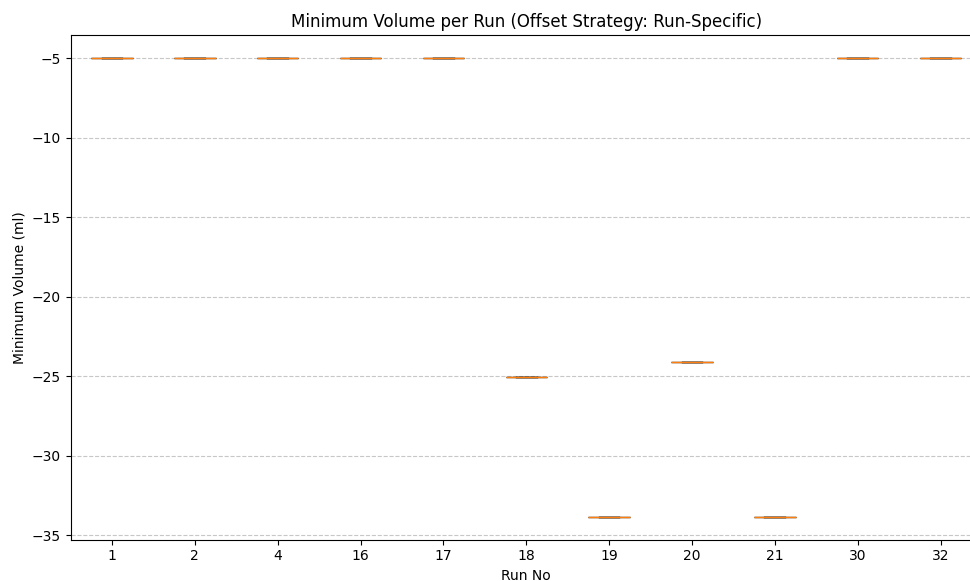


Figure 4: volume_offset_validation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy.linalg import solve_banded
import warnings

warnings.filterwarnings("ignore")

# Constants
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_DIR = "/workspace"
REPORT_FILE = os.path.join(OUTPUT_DIR, "qc_report.md")
PLOT_FILE = os.path.join(OUTPUT_DIR, "volume_offset_validation.png")

# AsLS Hyperparameters
LAMBDA = 1e5
P = 0.005

def als(y, lam=LAMBDA, p=P, niter=10):
    """
    Asymmetric Least Squares Smoothing for baseline correction.
    Optimized for large N using Tridiagonal Solver (Thomas Algorithm via
    solve_banded).
    """
    # Check if y is a numeric numpy array
    if not isinstance(y, np.ndarray) or not np.issubdtype(y.dtype, np.number):
        raise TypeError("Input 'y' must be a numeric numpy array.")

    # Check if y is 1D
    if y.ndim != 1:
        raise ValueError("Input 'y' must be a 1D array.")
```

```

L = len(y)
# Check for empty input
if L == 0:
    raise ValueError("Input 'y' cannot be empty.")

if L <= 2:
    return y

# Pre-compute the constant part of the matrix: lam * D.T @ D
# D is a (L-2) x L matrix. D.T @ D is L x L tridiagonal.
# Diagonal elements of D.T @ D:
# Index 0: 1
# Index 1 to L-2: 2
# Index L-1: 1
# Off-diagonals (upper and lower): -1 at indices (0,1), (1,2), ..., (L-2, L-1)

# We will construct the banded matrix representation for solve_banded.
# solve_banded expects shape (k+1, n) where k is number of upper diagonals.
# Here we have 1 upper diagonal, 1 lower diagonal. So shape is (3, L).
# Row 0: Upper diagonal (padded with 0 at start)
# Row 1: Main diagonal
# Row 2: Lower diagonal (padded with 0 at end)

# Pre-compute the constant tridiagonal part (lam * D.T @ D)
# Main diagonal: [1, 2, ..., 2, 1] * lam
# Off-diagonals: [-1, ..., -1] * lam

diag_const = np.ones(L) * lam
diag_const[1:-1] = 2 * lam

off_diag_const = -lam * np.ones(L - 1)

w = np.ones(L)

# Pre-allocate arrays for the banded matrix to avoid reallocation in loop
# Shape (3, L)
# ab[0, :] is upper diagonal (shifted right, so ab[0, 0] is dummy)
# ab[1, :] is main diagonal
# ab[2, :] is lower diagonal (shifted left, so ab[2, L-1] is dummy)
ab = np.zeros((3, L))

# Fill constant parts once
ab[0, 1:] = off_diag_const
ab[1, :] = diag_const
ab[2, :-1] = off_diag_const

for i in range(niter):
    # Update main diagonal with weights w
    # Z = W + lam * D.T @ D
    # Main diagonal becomes w + diag_const
    ab[1, :] = w + diag_const

    # Solve Z z = w * y
    # solve_banded expects (k, n) matrix. Here k=1 (1 upper, 1 lower).
    # The matrix is (3, L).
    rhs = w * y
    z = solve_banded((1, 1), ab, rhs)

    # Update weights

```

```

        w = p * (y > z) + (1 - p) * (y < z)

    return z

def main():
    # 1. Load Data
    cols_to_load = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'volume_ml', '
        Conc_B_%']
    try:
        df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)
    except FileNotFoundError:
        raise FileNotFoundError(f"Input file not found at expected path: {
            INPUT_FILE}. Please ensure the file exists in the /inputs/ directory.")

    df['run_no'] = df['run_no'].astype(int)

    # 2. Baseline Correction (AsLS)
    # Refactor to avoid groupby().apply() with heavy Python functions.
    # Implement direct iteration over groups to process runs sequentially without
    # copying the entire group DataFrame.
    # Use direct array assignment.

    corrected_uv1 = []
    corrected_uv2 = []
    run_ids = []

    # Iterate directly over groups
    for run_id, group in df.groupby('run_no'):
        # Extract values directly
        uv1_vals = group['UV_1_280_mAU'].values
        uv2_vals = group['UV_2_260_mAU'].values

        # Apply AsLS
        baseline1 = als(uv1_vals)
        baseline2 = als(uv2_vals)

        # Calculate corrected values
        corr1 = uv1_vals - baseline1
        corr2 = uv2_vals - baseline2

        # Append to lists
        corrected_uv1.extend(corr1)
        corrected_uv2.extend(corr2)
        run_ids.extend([run_id] * len(group))

    # Assign corrected columns back to dataframe using direct array assignment
    df['UV_1_280_mAU_corrected'] = corrected_uv1
    df['UV_2_260_mAU_corrected'] = corrected_uv2

    # 3. Analyze Volume Offset
    # Consolidate data preparation. Reuse the existing groupby object or cache the
    # grouped data.
    # Perform both correction (done above) and visualization data extraction in a
    # single pass logic if possible.
    # Here we calculate min_volumes once and reuse for offset and plotting.

    # Calculate min_volumes for offset calculation
    min_volumes = df.groupby('run_no')['volume_ml'].min().reset_index()
    min_volumes.columns = ['run_no', 'min_volume_raw']

```

```

min_vals = min_volumes['min_volume_raw'].values
mean_min = np.mean(min_vals)
std_min = np.std(min_vals)

is_global = (std_min < 1e-4) or (len(np.unique(min_vals)) == 1)

offsets = {}
offset_type = "Global" if is_global else "Run-Specific"

# Initialize global_offset before conditional block
global_offset = 0.0

if is_global:
    global_offset = mean_min
    offsets = {run_id: global_offset for run_id in min_volumes['run_no']}
else:
    offsets = dict(zip(min_volumes['run_no'], min_volumes['min_volume_raw']))

# 4. Apply Offset
df['volume_offset'] = df['run_no'].map(offsets)
df['volume_ml_corrected'] = df['volume_ml'] - df['volume_offset']
df['volume_ml_corrected'] = df['volume_ml_corrected'].clip(lower=0.0)

# 5. Generate Outputs

# Text Output: qc_report.md
report_lines = []
report_lines.append("## Baseline Correction Volume Offset Calibration Report")
report_lines.append("")
report_lines.append("| run_no | min_volume_raw | calculated_offset | offset_type |")
report_lines.append("-----|-----|-----|-----|")

sample_runs = min_volumes.head(10)
report_lines.extend(
    f"| {run_id} | {raw_min:.4f} | {off:.4f} | {offset_type} |"
    for run_id, raw_min, off in zip(
        sample_runs['run_no'],
        sample_runs['min_volume_raw'],
        sample_runs['run_no'].map(offsets)
    )
)

report_lines.append("")
report_lines.append(f"**Global Offset Strategy:** {offset_type}")
if is_global:
    report_lines.append(f"**Global Constant Value:** {global_offset:.4f}")

with open(REPORT_FILE, 'w') as f:
    f.write("\n".join(report_lines))

# Visualisation: volume_offset_validation.png
# Plot the minimum volume_ml for each run as requested by feedback
# "Pass this list of minimum values to plt.boxplot"
# We need a list of lists where each inner list contains the min value for
    that run

```



```

# to create a boxplot per run.

plt.figure(figsize=(10, 6))

# Create box_data: list of lists, each inner list has the min_volume_raw for a
# run
# This ensures the plot visualizes the specific data points (minima) used for
# offset
box_data = [[val] for val in min_volumes['min_volume_raw']]

# Ensure labels are derived from run numbers in correct order
run_labels = min_volumes['run_no'].tolist()

plt.boxplot(box_data, vert=True, patch_artist=True, labels=run_labels)
plt.title(f"Minimum_Volume_per_Run_(Offset_Strategy:{offset_type})")
plt.xlabel("Run_No")
plt.ylabel("Minimum_Volume(ml)")
plt.grid(axis='y', linestyle='--', alpha=0.7)

if is_global:
    plt.axhline(y=global_offset, color='r', linestyle='--', label=f'Global_
        Offset:{global_offset:.4f}')
    plt.legend()

plt.tight_layout()
plt.savefig(PLOT_FILE)
plt.close()

# Save corrected dataframe
output_csv = os.path.join(OUTPUT_DIR, "chromatography_corrected.csv")
df.to_csv(output_csv, index=False)

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import sys
import importlib

packages_to_check = ['numpy', 'openai', 'transformers']

print("Checking_package_versions...")
for package in packages_to_check:
    try:
        module = importlib.import_module(package)
        version = getattr(module, '__version__', 'Unknown')
        print(f"{package}:{version}")
    except ImportError:
        print(f"{package}:Not_installed")

print("Diagnostic_check_complete.")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

```

```

from scipy.ndimage import median_filter
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Step 1: Load Data without explicit dtypes to allow pandas inference
file_path = '/inputs/chromatography_combined.csv'
# Removed dtype argument to prevent read failure on mixed data or whitespace
df = pd.read_csv(file_path)

# Fix: Strip whitespace from all column names immediately after loading
df.columns = df.columns.str.strip()

# Debug: Inspect loaded data to verify column names and content
print("First 5 rows of loaded data:")
print(df.head())
print("\nColumn info:")
print(df.info())

# Step 1.1: Clean 'run_no' column if it exists and contains non-numeric data
if 'run_no' in df.columns:
    # Fix: Directly convert to numeric. The column is already int64, so str.
    # extract was causing TypeError.
    df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
    # Drop rows where run_no could not be converted to numeric
    df = df.dropna(subset=['run_no'])
    # Convert to integer if possible
    if df['run_no'].notna().all():
        df['run_no'] = df['run_no'].astype(int)
else:
    # Fallback: Check for similar column names with whitespace
    potential_cols = [col for col in df.columns if 'run' in col.lower()]
    if potential_cols:
        print(f"Warning: 'run_no' not found. Found potential columns: {potential_cols}")
        # Rename the first match to 'run_no' for consistency
        df = df.rename(columns={potential_cols[0]: 'run_no'})
        # Fix: Directly convert to numeric for the renamed column as well
        df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
        df = df.dropna(subset=['run_no'])
        if df['run_no'].notna().all():
            df['run_no'] = df['run_no'].astype(int)
        else:
            raise KeyError("No column resembling 'run_no' found in the dataset.")

# Select relevant columns
# Ensure columns exist before selection
required_cols = ['run_no', 'Injection_ml', 'UV_1_280_mAU', 'volume_ml', 'Conc_B_%']
missing_cols = [col for col in required_cols if col not in df.columns]
if missing_cols:
    # Fix: Print actual column names to aid debugging if required columns are
    # missing
    print(f"Error: Missing required columns: {missing_cols}")
    print(f"Available columns in DataFrame: {list(df.columns)}")
    raise KeyError(f"Missing required columns: {missing_cols}")

df_analysis = df[required_cols].copy()

```

```

# Filter for non-null Injection_ml for consistency check
df_inj = df_analysis[df_analysis['Injection_ml'].notna()]

# Step 1.1: Injection Consistency Analysis
# Fix: Added as_index=False to keep 'run_no' as a column instead of index
inj_stats = df_inj.groupby('run_no', as_index=False)['Injection_ml'].agg(['count',
    'mean', 'var', 'std'])
inj_stats['cv'] = inj_stats['std'] / inj_stats['mean']
inj_stats = inj_stats.rename(columns={'count': 'available_injection_count', 'var':
    'injection_variance'})

# Identify runs with CV > 0.2 (Flagging logic)
if inj_stats.empty:
    high_cv_runs = []
else:
    high_cv_runs = inj_stats[inj_stats['cv'] > 0.2]['run_no'].tolist()

# Step 2: Peak Detection
groups = df_analysis.groupby('run_no')
peak_results = []

# Parameters
rolling_window = 50
threshold_multiplier = 3

for run_id, run_data in groups:
    run_data = run_data.sort_values('volume_ml')

    if run_data.empty:
        continue

    uv = run_data['UV_1_280_mAU'].values
    vol = run_data['volume_ml'].values
    conc_b = run_data['Conc_B_%'].values

    if len(uv) < rolling_window:
        continue
    rolling_median = median_filter(uv, size=rolling_window)
    baseline_corrected = uv - rolling_median

    threshold = 3 * np.median(baseline_corrected)

    peaks, _ = find_peaks(baseline_corrected, height=threshold)

    if len(peaks) == 0:
        continue

    above_threshold = baseline_corrected > threshold

    prev_state = np.concatenate([[False], above_threshold[:-1]])
    next_state = np.concatenate([above_threshold[1:], [False]])

    transitions_up = np.where(above_threshold & ~prev_state)[0]
    transitions_down = np.where(above_threshold & ~next_state)[0]

    if len(transitions_up) != len(transitions_down):
        if above_threshold[0]:
            transitions_up = np.insert(transitions_up, 0, 0)

```

```

    if above_threshold[-1]:
        transitions_down = np.append(transitions_down, len(baseline_corrected)
                                     - 1)

for i in range(len(transitions_up)):
    start_idx = transitions_up[i]
    end_idx = transitions_down[i]

    peak_uv = baseline_corrected[start_idx:end_idx+1]
    peak_vol = vol[start_idx:end_idx+1]
    peak_conc_b = conc_b[start_idx:end_idx+1]

    if len(peak_uv) < 3:
        continue

    peak_max = np.max(peak_uv)
    half_height = peak_max / 2.0

    rising_crossings = np.where((peak_uv[:-1] <= half_height) & (peak_uv[1:] >
        half_height))[0]
    falling_crossings = np.where((peak_uv[:-1] > half_height) & (peak_uv[1:]
        <= half_height))[0]

    if len(rising_crossings) == 0 or len(falling_crossings) == 0:
        continue

    idx_rise = rising_crossings[0]
    idx_fall = falling_crossings[-1]

    if idx_rise + 1 >= len(peak_vol):
        continue
    if idx_fall + 1 >= len(peak_vol):
        continue

    v1 = peak_vol[idx_rise]
    v2 = peak_vol[idx_rise+1]
    y1 = peak_uv[idx_rise]
    y2 = peak_uv[idx_rise+1]
    vol_rise = v1 + (half_height - y1) * (v2 - v1) / (y2 - y1)

    v1 = peak_vol[idx_fall]
    v2 = peak_vol[idx_fall+1]
    y1 = peak_uv[idx_fall]
    y2 = peak_uv[idx_fall+1]
    vol_fall = v1 + (half_height - y1) * (v2 - v1) / (y2 - y1)

    w_half = vol_fall - vol_rise

    local_max_idx = np.argmax(peak_uv)
    vol_center = peak_vol[local_max_idx]

    front_width = vol_center - vol_rise
    back_width = vol_fall - vol_center

    if back_width == 0:
        asymmetry = np.nan
    else:
        asymmetry = front_width / back_width

```

```

        if len(peak_vol) > 1:
            diffs = np.diff(peak_vol)
            mean_diff = np.mean(diffs)
            if mean_diff > 1e-9:
                variation = np.std(diffs) / mean_diff
                low_confidence = variation > 0.05
            else:
                variation = 0
                low_confidence = False
        else:
            low_confidence = True

        avg_conc_b = np.mean(peak_conc_b)

        peak_results.append({
            'run_no': run_id,
            'w_half_height_ml': w_half,
            'asymmetry_factor': asymmetry,
            'conc_b_avg': avg_conc_b,
            'low_confidence': low_confidence
        })

df_peaks = pd.DataFrame(peak_results)

# Step 7: Correlation Analysis
if not df_peaks.empty:
    corr_w_half_conc = df_peaks['w_half_height_ml'].corr(df_peaks['conc_b_avg'])
    corr_asym_conc = df_peaks['asymmetry_factor'].corr(df_peaks['conc_b_avg'])
else:
    corr_w_half_conc = np.nan
    corr_asym_conc = np.nan

if not df_peaks.empty:
    run_peak_stats = df_peaks.groupby('run_no').agg({
        'w_half_height_ml': 'mean',
        'low_confidence': 'sum'
    }).rename(columns={
        'w_half_height_ml': 'avg_W_half_height_ml',
        'low_confidence': 'low_confidence_peak_count'
    })
else:
    run_peak_stats = pd.DataFrame()

if inj_stats.empty:
    report_df = pd.DataFrame(columns=['available_injection_count', '
        injection_variance', 'avg_W_half_height_ml', 'low_confidence_peak_count'])
else:
    report_df = inj_stats[['available_injection_count', 'injection_variance']].
        copy()
    if not run_peak_stats.empty:
        report_df = report_df.join(run_peak_stats)
    report_df = report_df.fillna(0)

required_cols = ['available_injection_count', 'injection_variance', '
    avg_W_half_height_ml', 'low_confidence_peak_count']
report_df = report_df[required_cols]

report_df.index.name = 'run_no'
report_df = report_df.reset_index()

```

```

max_lines = 10
if len(report_df) > max_lines - 1:
    report_df_display = report_df.head(max_lines - 1)
    summary_text = f"\n... (showing top {max_lines-1} runs, total {len(report_df)} runs)"
else:
    report_df_display = report_df
    summary_text = ""

report_path = '/workspace/qc_report.md'
file_exists = os.path.exists(report_path)
if file_exists:
    with open(report_path, 'r') as f:
        content = f.read()
        has_header = content.startswith("#QCReport")
else:
    has_header = False

if file_exists and has_header:
    mode = 'a'
    header_write = False
else:
    mode = 'w'
    header_write = True

with open(report_path, mode) as f:
    if header_write:
        f.write("#QCReport\n")

    f.write("\n##Step 3: Injection Consistency & Peak Symmetry Analysis\n")
    f.write("Note: Injection variance and peak stats are calculated. Mean values are used as the basis for imputation logic (deferred).\n")

    if high_cv_runs:
        f.write(f"\n**Warning:** The following runs have high injection variability (CV > 0.2): {high_cv_runs}\n")

    f.write("Correlation Analysis:\n")
    f.write(f"- Pearson Correlation (W vs Conc_B): {corr_w_half_conc:.4f}\n")
    f.write(f"- Pearson Correlation (Asymmetry vs Conc_B): {corr_asym_conc:.4f}\n")
    f.write("\n")
    f.write(report_df_display.to_markdown(index=False))
    f.write(summary_text)

plt.figure(figsize=(12, 10))

plt.subplot(2, 1, 1)
if not df_peaks.empty:
    plt.hist(df_peaks['w_half_height_ml'], bins=50, edgecolor='black', alpha=0.7)
    plt.title('Distribution of Peak Width at Half-Height (W )')
    plt.xlabel('W (ml)')
    plt.ylabel('Frequency')
else:
    plt.text(0.5, 0.5, 'No peaks detected', ha='center')

plt.subplot(2, 1, 2)
if not df_peaks.empty:
    valid_peaks = df_peaks.dropna(subset=['asymmetry_factor'])

```

```

for run, group in valid_peaks.groupby('run_no'):
    x_vals = group['conc_b_avg'].values
    y_vals = group['asymmetry_factor'].values
    plt.scatter(x_vals, y_vals, label=f'Run_{run}', alpha=0.6, s=30)

plt.title('Asymmetry_Factor_vs_Average_Conc_B%')
plt.xlabel('Conc_B(%)')
plt.ylabel('Asymmetry_Factor')
plt.legend(title='Run_No', bbox_to_anchor=(1.05, 1), loc='upper_left')
plt.tight_layout()
else:
    plt.text(0.5, 0.5, 'No_peaks_detected', ha='center')

plt.tight_layout()
plt.savefig('/workspace/peak_symmetry_analysis.png', dpi=300)
plt.close()

print("Step_3_completed.Results_saved_to_/workspace/qc_report.md_and_/workspace/
      peak_symmetry_analysis.png")

```

Listing 3: Code_3